QUALCOMM AI RESEARCH (QARE) PREP • EYE-CARE WARM PARCHMENT EDITION

Qualcomm AI Residency (QARE) — Phase 1 Master Study Plan

14-Day Day-by-Day Roadmap, Topic Checklist & Exam Execution Strategy

INTERACTIVE DIRECTORY • CLICK ANY TOPIC TO JUMP

Table of Contents (Mục Lục)

11 Sections Indexed

1. The HackerRank MCQ Meta-Game: How Questions Are Built ..... p. 2
   - Golden Rules for Trắc Nghiệm Preparation ..... p. 2
2. Pillar-by-Pillar Core Curriculum & High-Yield Formulas ..... p. 3
   - Pillar 1: Mathematics for AI (Linear Algebra, Calculus, Probability & Stats) ..... p. 3
   - Pillar 2: Classical Machine Learning & Deep Learning Fundamentals ..... p. 5
   - Pillar 3: Computer Vision (CV) ..... p. 7
   - Pillar 4: Natural Language Processing (NLP) ..... p. 8
   - Pillar 5: Large Language Models (LLMs), Transformers & Efficient AI ..... p. 9
3. Personalized 20-Day Execution Calendar (Oct 29 – Nov 18, 2026) ..... p. 12
   - Day-by-Day Master Schedule (Oct 29 – Nov 18) ..... p. 12
4. Active `QARE_prep` Workspace Architecture ..... p. 15

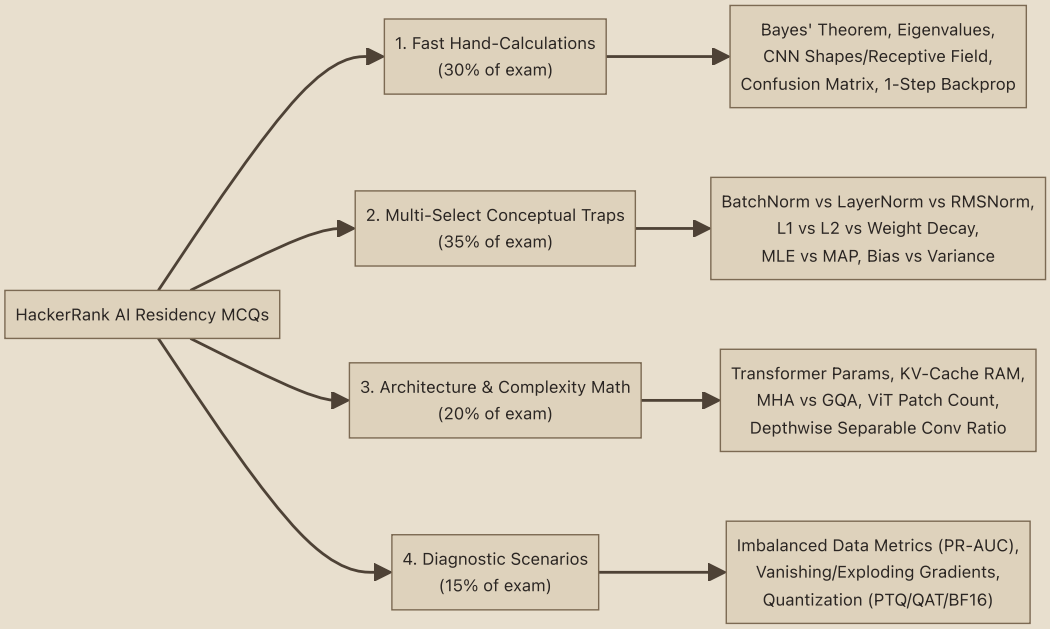
> ⚡ **IMPORTANT**
>
> **Phase 1 Objective:** Dominate the **HackerRank Multiple-Choice (Trắc Nghiệm) Entrance Test** across **Mathematics, Classical ML, Deep Learning, Computer Vision (CV), NLP, and LLMs/Efficient AI**. Because HackerRank assessments are strictly timed and proctored, passive reading is not enough—you must master **90-second hand calculations**, **exact tensor/memory formulas**, and "**Select All That Apply**" **conceptual traps**.

---

1. The HackerRank MCQ Meta-Game: How Questions Are Built

↑ Mục Lục

Qualcomm AI Residency (Vietnam) entrance tests evaluate whether you understand **first-principles mechanics** rather than high-level API calls. Almost every MCQ on the exam falls into one of four archetypes:



Golden Rules for Trắc Nghiệm Preparation

1. **Never derive from scratch what you can compute with an invariant:** For example, never expand a 3×3 characteristic polynomial $\det(A - \lambda I) = 0$ if you can eliminate 3 out of 4 MCQ options in 10 seconds using $\sum \lambda_i = \text{Tr}(A)$ and $\prod \lambda_i = \det(A)$.
2. **Watch out for Multi-Select ("Select all that apply"):** HackerRank awards partial or zero credit if you miss a subtle edge case (e.g., "*Does BatchNorm have learnable parameters?*" $\rightarrow$ Yes, γ and β, $2C$ parameters total; "*Does a Conv layer before BatchNorm need a bias term?*" $\rightarrow$ No, BatchNorm's mean subtraction cancels it out).
3. **Know Qualcomm's Engineering DNA:** As an **Engineering Track** candidate, expect extra emphasis on **computational efficiency, FLOPs/MACs, parameter counts, memory footprint (KV-cache, mixed precision FP16/BF16), quantization (INT8/INT4), and mobile/edge architectures (MobileNet, GQA, LoRA, Knowledge Distillation)**.

---

2. Pillar-by-Pillar Core Curriculum & High-Yield Formulas

↑ Mục Lục

Below is the prioritized syllabus focusing strictly on the **foundational, highest-frequency concepts** tested in AI Residency entrance exams.

Pillar 1: Mathematics for AI (Linear Algebra, Calculus, Probability & Stats)

1.1 Linear Algebra (High-Speed Matrix Mechanics)

- **Eigenvalues & Eigenvectors ($Av = \lambda v$):**
  - **Speed Hacks:** $\text{Tr}(A) = \sum_{i=1}^{n} A_{ii} = \sum_{i=1}^{n} \lambda_i$ and $\det(A) = \prod_{i=1}^{n} \lambda_i$.
  - ** Eigenvalue Shift & Scaling:**
    - Eigenvalues of $(A + cI)$ are $\lambda_i + c$.
    - Eigenvalues of A^k are λ_i^k.
    - Eigenvalues of A^{-1} are $1/\lambda_i$.
- **Rank, Nullity & Linear Systems ($Ax = b$):**
  - **Rank-Nullity Theorem:** For $A \in \mathbb{R}^{m \times n}$, $\text{rank}(A) + \dim(\text{Null}(A)) = n$ (number of columns).
  - $\text{rank}(AB) \leq \min(\text{rank}(A), \text{rank}(B))$, and $\text{rank}(A^T A) = \text{rank}(AA^T) = \text{rank}(A)$.
  - System $Ax = b$ has:
    - **0 solutions** if $\text{rank}(A) < \text{rank}([A \mid b])$.
    - **1 unique solution** if $\text{rank}(A) = \text{rank}([A \mid b]) = n$.
    - ∞ **solutions** if $\text{rank}(A) = \text{rank}([A \mid b]) < n$.
- **Symmetric & Positive Semi-Definite (PSD) Matrices:**
  - Spectral Theorem: Any real symmetric matrix ($A = A^T$) has **all real eigenvalues** and **orthogonal eigenvectors** ($A = Q\Lambda Q^T$ with $Q^T Q = I$).
  - $A \succeq 0$ (PSD) $\iff \forall x, x^T Ax \geq 0 \iff$ all $\lambda_i \geq 0$. Note: Gram/Covariance matrices $X^T X$ are **always PSD**.
- **SVD ($A = U\Sigma V^T$) & PCA:**
  - Left singular vectors U = eigenvectors of AA^T; Right singular vectors V = eigenvectors of $A^T A$; Singular values $\sigma_i = \sqrt{\lambda_i(A^T A)}$.
  - **Eckart-Young Theorem:** Best rank-k approximation of A in Frobenius or L_2 norm is $A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^T$.
  - **PCA Variance Explained:** Ratio for top k components is $\frac{\sum_{i=1}^{k} \lambda_i}{\sum_{j=1}^{d} \lambda_j}$. Data **must be mean-centered** before computing covariance $C = \frac{1}{N-1} X^T X$.
- **Vector Norms & Projections:**
  - L_1 norm $\|x\|_1 = \sum |x_i|$ (diamond unit ball), L_2 norm $\|x\|_2 = \sqrt{\sum x_i^2}$ (sphere), L_∞ norm $\|x\|_\infty = \max_i |x_i|$ (cube).
  - Orthogonal projection of vector y onto vector u: $\text{proj}_u(y) = \frac{u^T y}{\|u\|_2^2} u$.
  - Projection matrix onto column space of A: $P = A(A^T A)^{-1} A^T$. Properties: $P^T = P$ (symmetric) and $P^2 = P$ (idempotent $\implies$ eigenvalues are only 0 or 1).

1.2 Multivariable Calculus & Optimization

- **Essential Vector/Matrix Gradients (Memorize Cold):**
  - $\nabla_x(a^T x) = a$
  - $\nabla_x(x^T A x) = (A + A^T)x$ (simplifies to $2Ax$ when A is symmetric)
  - $\nabla_w \|Xw - y\|_2^2 = 2X^T(Xw - y)$
- **Clean Loss + Activation Gradients (Classic Backprop MCQ):**
  - **Sigmoid:** $\sigma(z) = \frac{1}{1+e^{-z}} \implies \sigma'(z) = \sigma(z)(1 - \sigma(z))$. Maximum derivative is 0.25 at $z = 0$ (root cause of vanishing gradients in deep sigmoid nets).
  - **Tanh:** $\tanh'(z) = 1 - \tanh^2(z)$. Maximum derivative is 1.0 at $z = 0$, zero-centered output $\in (-1, 1)$.
  - **Binary Cross-Entropy + Sigmoid** OR **Multiclass Cross-Entropy + Softmax:** The gradient with respect to the pre-activation logits z simplifies to:

$$\frac{\partial \mathcal{L}}{\partial z} = \hat{y} - y$$

- **Second-Order Conditions & Convexity:**
  - At critical point $\nabla f(x^*) = 0$: Hessian $H \succ 0 \implies$ strict local minimum; $H \prec 0 \implies$ strict local maximum; H has both positive and negative eigenvalues $\implies$ **saddle point**.
  - **Convex Function Definitions:**
    1. $f(\theta x + (1-\theta)y) \leq \theta f(x) + (1-\theta)f(y)$ for $\theta \in [0, 1]$ (Jensen's Inequality: $\mathbb{E}[f(X)] \geq f(\mathbb{E}[X])$).
    2. First-order condition: $f(y) \geq f(x) + \nabla f(x)^T(y - x)$ (tangent plane is always a global underestimator).
    3. Second-order condition: $\nabla^2 f(x) \succeq 0$ everywhere.
- **Gradient Descent & Modern Optimizers:**
  - Exact convergence condition for GD $x_{t+1} = x_t - \eta \nabla f(x_t)$ on L-smooth convex function: step size $0 < \eta < \frac{2}{L}$ (where $L = \lambda_{\max}(H)$).
  - **Adam Update Rules:** Tracks 1st moment $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ and 2nd uncentered moment $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$.
    - *MCQ Trap:* Why does Adam divide by $1 - \beta_1^t$ and $1 - \beta_2^t$? Because $m_0 = 0, v_0 = 0$ biases early estimates toward zero!
    - *Adam vs AdamW:* For adaptive optimizers like Adam, adding $\frac{\lambda}{2}\|w\|_2^2$ to the loss (L_2 regularization) is **not equivalent** to decoupled weight decay because the regularization gradient gets scaled by $1/\sqrt{\hat{v}_t}$. **AdamW** applies weight decay directly to weights ($w_t \leftarrow (1 - \eta\lambda)w_t$).

1.3 Probability, Statistics & Information Theory

- **Bayes' Theorem & Diagnostic Testing Paradox:**

$$P(D \mid +) = \frac{P(+ \mid D)P(D)}{P(+ \mid D)P(D) + P(+ \mid \neg D)P(\neg D)}$$

  - Know the terminology: **Sensitivity (Recall / TPR)** $= P(+ \mid D)$; **Specificity (TNR)** $= P(- \mid \neg D) = 1 -$ FPR.

- **Expectation, Variance & Covariance Rules:**
  - $\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$ (holds **even if X, Y are dependent!**).
  - $\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$.
  - $\text{Var}(aX + bY) = a^2\text{Var}(X) + b^2\text{Var}(Y) + 2ab\,\text{Cov}(X, Y)$.
  - *Trap:* Uncorrelated ($\text{Cov}(X, Y) = 0$) does **not** imply independent (unless jointly Gaussian), e.g., $X \sim \mathcal{N}(0, 1)$ and $Y = X^2$ have $\text{Cov}(X, Y) = 0$ but are completely dependent!
- **Maximum Likelihood (MLE) vs Maximum A Posteriori (MAP):**
  - $\hat{\theta}_{\text{MLE}} = \arg\max_\theta \sum_i \log P(x_i \mid \theta)$.
  - $\hat{\theta}_{\text{MAP}} = \arg\max_\theta \left[\sum_i \log P(x_i \mid \theta) + \log P(\theta)\right]$.
  - **Fundamental Equivalences (Guaranteed MCQ):**
    - Linear Regression with i.i.d. Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma^2)$ $\iff$ **Minimizing MSE**.
    - MAP with **Gaussian prior** $w \sim \mathcal{N}(0, \tau^2 I)$ $\iff$ **L2 Regularization (Ridge)**.
    - MAP with **Laplace prior** $w \sim \text{Laplace}(0, b)$ $\iff$ **L1 Regularization (Lasso)**.
- **Information Theory:**
  - Entropy: $H(P) = -\sum_x P(x) \log P(x)$ (maximized when P is **uniform**, where $H = \log K$).
  - Cross-Entropy: $H(P, Q) = -\sum_x P(x) \log Q(x) = H(P) + D_{\text{KL}}(P \parallel Q)$.
  - KL Divergence: $D_{\text{KL}}(P \parallel Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)} \geq 0$. **Asymmetric** ($D_{\text{KL}}(P \parallel Q) \neq D_{\text{KL}}(Q \parallel P)$), so not a true metric. Minimizing Cross-Entropy w.r.t. model Q_θ is identical to minimizing $D_{\text{KL}}(P_{\text{data}} \parallel Q_\theta)$ because $H(P_{\text{data}})$ is constant w.r.t. θ.

Pillar 2: Classical Machine Learning & Deep Learning Fundamentals

2.1 Classical ML Workhorses

Algorithm	Core Mathematical Mechanism	Classic HackerRank MCQ Traps
Linear / Ridge Regression	Closed-form: $w = (X^T X + \lambda I)^{-1} X^T y$	Adding λI ($\lambda > 0$) makes $X^T X + \lambda I$ strictly positive definite and **always invertible**, even if features are collinear ($d > N$).
Logistic Regression	$P(y = 1 \mid x) = \sigma(w^T x + b)$; Convex BCE loss	Decision boundary $w^T x + b = 0$ is **linear**. If data is linearly separable without regularization, $\|w\| \to \infty$.
SVM (Hard & Soft Margin)	$\min_{w,b,\xi} \frac{1}{2}\|w\|_2^2 + C\sum \xi_i$ s.t. $y_i(w^T x_i + b) \geq 1 - \xi_i$	Distance between margin hyperplanes is $\frac{2}{\|w\|_2}$. **Large** C = heavy penalty on slack $\implies$ narrow margin, harder fit (high variance). Only **Support Vectors** have $\alpha_i > 0$.
Decision Trees	Splits maximize Information Gain ΔH or Gini ΔG ($G = 1 - \sum p_k^2$)	Scale-invariant (do **not** need feature standardization). Prone to high variance when deep.
Random Forest (Bagging)	Bootstrap samples + random feature subset per split	Primarily reduces **variance**. Out-of-Bag (OOB) sample ratio approaches $1/e \approx 36.8\%$ as $N \to \infty$.
Gradient Boosting (XGBoost)	Sequential trees fit negative gradients (pseudo-residuals)	Primarily reduces **bias**. Can overfit if number of trees M is too large (unlike Random Forest).

Algorithm	Core Mathematical Mechanism	Classic HackerRank MCQ Traps
K-Means Clustering	Alternates cluster assignment & centroid mean update	Minimizes Within-Cluster Sum of Squares (non-convex). Fails on non-spherical/concentric clusters; sensitive to feature scale and outliers.

2.2 Evaluation Metrics & Bias-Variance Diagnostics

- **Confusion Matrix Quick Reference:**
  - Precision $= \frac{TP}{TP+FP}$ ("*Of predicted positives, how many are truly positive?*")
  - Recall (Sensitivity / TPR) $= \frac{TP}{TP+FN}$ ("*Of actual positives, how many did we catch?*")
  - Specificity (TNR) $= \frac{TN}{TN+FP}$; FPR $= 1 -$ Specificity $= \frac{FP}{FP+TN}$
  - F1-Score $= \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision}+\text{Recall}} = \frac{2TP}{2TP+FP+FN}$ (Harmonic mean — heavily penalized if either P or R is low).
- **ROC-AUC vs PR-AUC on Imbalanced Datasets:**
  - ROC plots **TPR vs FPR**. When negatives (TN) are massive (e.g., 99.9% negative fraud detection), a large surge in FP barely changes FPR $= \frac{FP}{FP+TN}$, making ROC-AUC look deceptively high.
  - **PR-AUC** (Precision vs Recall) does not use TN at all and is the **correct choice** for rare positive classes.
- **L1 vs L2 Sparsity Geometry:**
  - L1 penalty $\lambda\|w\|_1$ has constant gradient $\pm\lambda$ all the way to 0 (and subgradient $[-\lambda, \lambda]$ at 0), driving small weights to **exact zero** (feature selection).
  - L2 penalty $\frac{\lambda}{2}\|w\|_2^2$ has gradient λw, which shrinks proportionally as $w \to 0$ and rarely hits exact zero.

2.3 Deep Learning Layers & Training Mechanics

- **Normalization Layers Comparison (Top-Tier Exam Favorite):**

Layer	Normalization Axes (Input N, C, H, W or N, L, D)	Train vs Inference Behavior	Learnable Parameters	Primary Use Case
BatchNorm	Across (N, H, W) per channel C	Uses batch stats in train; **EMA running mean/var** in eval	$2C$ (γ, β); plus $2C$ non-trainable running buffers	CNNs with reasonable batch size
LayerNorm	Across (C, H, W) or hidden dim D per token	**Identical** in train and eval (computes per-sample stats on the fly)	$2D$ (γ, β)	Transformers / LLMs / RNNs
RMSNorm	Across hidden dim D per token (no mean centering: $\frac{x}{\sqrt{\frac{1}{D}\sum x_i^2+\epsilon}} \odot \gamma$)	**Identical** in train and eval	D (γ only, no β)	Modern LLMs (LLaMA, Qwen, Mistral)
GroupNorm	Splits C into G groups, normalizes $(C/G, H, W)$ per sample	**Identical** in train and eval	$2C$ (γ, β)	Object Detection / Small-batch CV

- **Inverted Dropout:**
  - During training, zeroes activations with probability p and **scales remaining activations by** $\frac{1}{1-p}$ so that $\mathbb{E}[a_{\text{train}}] = a_{\text{test}}$. At test time, Dropout is an identity pass-through (`model.eval()`).

- **Weight Initialization:**
  - *Why can't weights W be initialized to all zeros?* Every neuron in a layer computes the exact same output and receives the exact same gradient (**symmetry breaking failure**). Biases b, however, *should* typically be initialized to 0.
  - **Xavier/Glorot:** $\text{Var}(W) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$ (designed for linear/Tanh/Sigmoid).
  - **Kaiming/He:** $\text{Var}(W) = \frac{2}{n_{\text{in}}}$ (accounts for ReLU zeroing out half the distribution).

Pillar 3: Computer Vision (CV)

3.1 Convolution, Pooling & Receptive Field Formulas

- **2D Convolution / Pooling Output Dimension:**

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P - D \times (K-1) - 1}{S} \right\rfloor + 1$$

(When dilation $D = 1$, this simplifies to $\lfloor \frac{H_{\text{in}} - K + 2P}{S} \rfloor + 1$. For "same" padding with $S = 1, D = 1$, set $P = \frac{K-1}{2}$.)

- **Parameter Count & MACs/FLOPs of Standard Conv2D:**
  - $\text{Params} = (K_h \times K_w \times C_{\text{in}} + \mathbb{I}_{\text{bias}}) \times C_{\text{out}}$
  - $\text{MACs} = H_{\text{out}} \times W_{\text{out}} \times K_h \times K_w \times C_{\text{in}} \times C_{\text{out}}$ *(Note:* 1 MAC $\approx$ 2 FLOPs*).*
- **Grouped & Depthwise Separable Convolutions (MobileNet / Edge AI):**
  - **Grouped Conv (G groups):** $\text{Params} = \frac{1}{G}(K^2 C_{\text{in}} C_{\text{out}})$.
  - **Depthwise Separable Conv:** Factorizes standard Conv into:
    1. **Depthwise Conv ($G = C_{\text{in}}$):** $K \times K \times C_{\text{in}}$ parameters.
    2. **Pointwise Conv (1×1):** $1 \times 1 \times C_{\text{in}} \times C_{\text{out}}$ parameters.

    - **Total Params:** $K^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}}$.
    - **Reduction Ratio vs Standard Conv:** $\frac{K^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}}}{K^2 C_{\text{in}} C_{\text{out}}} = \frac{1}{C_{\text{out}}} + \frac{1}{K^2}$ *(For 3×3 kernels, $\approx$ 8–9$\times$ fewer parameters and MACs!).*
- **Receptive Field Calculation (Iterative Formula from Input to Layer L):**
  - Maintain cumulative stride $J_l = J_{l-1} \times S_l$ (with $J_0 = 1$) and receptive field RF_l:

$$RF_l = RF_{l-1} + (K_l - 1) \times J_{l-1} \quad (\text{with } RF_0 = 1)$$

  - *Classic Shortcut:* Stacking two 3×3 stride-1 convolutions gives an effective receptive field of 5×5 with $2 \times (3^2 C^2) = 18C^2$ parameters (compared to $25C^2$ for a single 5×5 conv), while adding an extra non-linearity! Stacking three 3×3 stride-1 convolutions gives a 7×7 receptive field ($27C^2$ vs $49C^2$).

3.2 Architectures, Detection, Segmentation & Vision Transformers

- **ResNet & Skip Connections ($y = \mathcal{F}(x) + x$):**
  - Solves the **degradation problem** (where deeper plain networks have *higher training error*—note: degradation is NOT overfitting!).

- Backward pass: $\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y}\left(I + \frac{\partial \mathcal{F}(x)}{\partial x}\right)$ allows gradients to flow unimpeded through the identity path.
- **Bottleneck Block (ResNet-50+):** 1×1 (reduce channels $256 \to 64$) $\to 3 \times 3$ ($64 \to 64$) $\to 1 \times 1$ (expand $64 \to 256$).

- **Object Detection Essentials:**
  - **IoU:** $\frac{\text{Area}(B_1 \cap B_2)}{\text{Area}(B_1 \cup B_2)} = \frac{\text{Intersection}}{\text{Area}(B_1)+\text{Area}(B_2)-\text{Intersection}}$.
  - **Non-Maximum Suppression (NMS):**
    1. Discard boxes with confidence score $< \theta_{\text{conf}}$.
    2. Pick highest-scoring box $B_{\max}$, add to output.
    3. Remove all remaining boxes of the **same class** with $\text{IoU}(B_i, B_{\max}) > \theta_{\text{IoU}}$.
    4. Repeat until no boxes remain.
  - **Focal Loss (RetinaNet):** $\text{FL}(p_t) = -\alpha_t(1-p_t)^\gamma \log(p_t)$. The modulating factor $(1-p_t)^\gamma$ down-weights **easy examples** ($p_t \to 1$) so training focuses on hard negatives/positives in one-stage detectors.
  - **RoI Pooling vs RoI Align (Mask R-CNN):** RoI Pooling quantizes floating-point box coordinates to discrete grid cells (causing spatial misalignment for pixel masks); **RoI Align** removes quantization and uses **bilinear interpolation** at exact sub-pixel sample points.

- **Vision Transformer (ViT) Mechanics:**
  - Input image $H \times W \times C$, patch size $P \times P \implies$ number of patches $N = \frac{H \cdot W}{P^2}$ (e.g., 224×224 with $P = 16 \implies N = 14 \times 14 = 196$ patches).
  - Each patch is flattened to P^2C (e.g., $16^2 \times 3 = 768$) and linearly projected to D.
  - Prepend 1 learnable `[CLS]` token $\implies$ sequence length entering Transformer Encoder is $N+1$ (e.g., $197 \times D$), added element-wise to a learnable $(N+1) \times D$ positional embedding.
  - *CNN vs ViT Inductive Bias:* CNNs have strong built-in **locality** and **2D translation equivariance**; ViTs have **weak inductive bias** and global self-attention from layer 1 (higher capacity on massive datasets, worse sample efficiency on small datasets without pretraining).

Pillar 4: Natural Language Processing (NLP)

4.1 Subword Tokenization & Classical Representations

- **Subword Tokenization Algorithms (Know the Exact Merge Criterion):**
  - **BPE (Byte-Pair Encoding — GPT/LLaMA):** Iteratively merges the **most frequent adjacent pair** of tokens: $\arg\max_{(a,b)} \text{count}(ab)$.
  - **WordPiece (BERT):** Iteratively merges the pair that maximizes **language model likelihood / pointwise mutual information**: $\arg\max_{(a,b)} \frac{\text{count}(ab)}{\text{count}(a)\text{count}(b)}$.
  - **Unigram LM (SentencePiece / T5):** Starts with a large vocabulary and iteratively prunes tokens that minimally decrease corpus likelihood under a unigram language model.

- **TF-IDF:**
  - $\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \log\left(\frac{|D|}{1+|\{d \in D : t \in d\}|}\right)$.
  - High when a word occurs **frequently in document** d but **rarely across the entire corpus** D. Stop words ("the", "is") get IDF ≈ 0.

4.2 Word Embeddings, Sequence Models & NLP Metrics

- **Word2Vec (Skip-gram vs CBOW):**
  - **CBOW:** Predicts center word from sum/average of surrounding context words (faster to train, better for frequent words).
  - **Skip-gram:** Predicts surrounding context words given center word (slower, works much better for **rare words**).
  - **Negative Sampling (SGNS):** Replaces expensive $O(|V|)$ full softmax normalization with k binary logistic regressions distinguishing the true target word from k noise words sampled from $P_n(w) \propto U(w)^{3/4}$.
- **RNNs, LSTMs & GRUs:**
  - Why do vanilla RNNs $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$ suffer from vanishing/exploding gradients? Backprop through time (BPTT) multiplies $\prod_{k=1}^{T} \frac{\partial h_k}{\partial h_{k-1}} \propto W_{hh}^T$. If the largest singular value $\sigma_{\max}(W_{hh}) < 1$, gradients vanish exponentially; if > 1, gradients explode (**remedy for exploding gradients: Gradient Clipping** by global norm).
  - **LSTM Cell State Highway:** $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$. The **additive** update modulated by the forget gate $f_t \in (0, 1)$ allows error gradients $\frac{\partial c_t}{\partial c_{t-1}} \approx f_t$ to propagate across long sequences without vanishing.
  - **Parameter Counts (for input dim $x \in \mathbb{R}^d$ and hidden state $h \in \mathbb{R}^h$):**
    - Vanilla RNN: $1 \times (h(d + h) + h)$
    - GRU (3 gates: reset, update, candidate): $3 \times (h(d + h) + h)$ (or $3(hd + h^2 + 2h)$ in PyTorch implementation)
    - LSTM (4 gates: input, forget, cell candidate, output): $4 \times (h(d + h) + h)$
- **Decoding & Generation Metrics:**
  - **Perplexity:** $\text{PPL} = \exp\left(-\frac{1}{T}\sum_{t=1}^{T} \log P(w_t \mid w_{<t})\right) = \exp(\text{Cross-Entropy Loss in nats})$. If a model assigns uniform probability over V words, $\text{PPL} = V$. Lower is better.
  - **BLEU:** Precision-oriented n-gram overlap with **clipped counts** + **Brevity Penalty (BP)** to penalize overly short candidate translations.
  - **ROUGE:** Recall-oriented n-gram/LCS overlap (standard for summarization).

Pillar 5: Large Language Models (LLMs), Transformers & Efficient AI

> **TIP**
>
> **Why Pillar 5 Is Crucial for Qualcomm Engineering Track:** Qualcomm's AI Residency Engineering Track heavily emphasizes **GenAI, On-Device LLMs, Transformer internals, KV-Cache optimization, LoRA, and Quantization**. Expect detailed architectural and numerical questions here.

5.1 Transformer Architecture & Attention Math

- **Scaled Dot-Product Attention:**

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

- **Why scale by $\frac{1}{\sqrt{d_k}}$?** Assume components of $q, k \in \mathbb{R}^{d_k}$ are independent random variables with mean 0 and variance 1. Then their dot product $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ has mean 0 and **variance** d_k (standard deviation $\sqrt{d_k}$). For large d_k, unscaled dot products grow large in magnitude, pushing softmax into regions with near-zero gradients (softmax $\rightarrow$ one-hot). Dividing by $\sqrt{d_k}$ restores **unit variance (**$\text{Var} = 1$**)**.
- **Causal Mask M:** In autoregressive decoders (GPT/LLaMA), upper-triangular entries ($j > i$) are set to $-\infty$ *before* the softmax so that $\exp(-\infty) = 0$ (preventing token i from attending to future tokens $j > i$).

- **Computational Complexity of Self-Attention (Sequence length N, Model dim D):**
  - QK^T: multiplying $(N \times d_k)$ by $(d_k \times N)$ across h heads takes $O(N^2 D)$ time and $O(N^2)$ attention matrix memory (unless using **FlashAttention**, which tiles Q, K, V in SRAM using online softmax to achieve $O(N)$ **HBM memory** with exact same output!).
  - Linear Projections (W_Q, W_K, W_V, W_O): take $O(ND^2)$ time.
  - *Crossover Rule:* When sequence length $N \ll D$, projection layers ($O(ND^2)$) dominate; when $N \gg D$ (long context), quadratic attention ($O(N^2 D)$) dominates.

- **Exact Transformer Block Parameter Count (Standard vs Modern LLaMA):**
  - **Multi-Head Attention (MHA):** $W_Q, W_K, W_V, W_O \in \mathbb{R}^{D \times D} \implies \mathbf{4D^2}$ parameters (independent of the number of heads h, since $h \times d_k = D$!).
  - **Standard FFN (2 matrices, intermediate dim $4D$):** $W_1 \in \mathbb{R}^{D \times 4D}, W_2 \in \mathbb{R}^{4D \times D} \implies \mathbf{8D^2}$ parameters. Total per standard Transformer layer $\approx \mathbf{12D^2}$.
  - **SwiGLU FFN (LLaMA/Mistral — 3 matrices: Gate, Up, Down with intermediate dim D_{ff}):** $\text{SwiGLU}(x) = (\text{Swish}(xW_{\text{gate}}) \odot xW_{\text{up}})W_{\text{down}} \implies \mathbf{3D \cdot D_{\text{ff}}}$ parameters (typically $D_{\text{ff}} \approx \frac{8}{3}D$ so total FFN params remain $\approx 8D^2$).

- **MHA vs MQA vs GQA (KV-Cache Optimization):**
  - **MHA (Multi-Head Attention):** h query heads, h key heads, h value heads.
  - **MQA (Multi-Query Attention):** h query heads share 1 **single key head and** 1 **single value head** (reduces KV-cache by $h\times$, slight quality drop).
  - **GQA (Grouped-Query Attention — LLaMA-2 70B, LLaMA-3):** h query heads are partitioned into g groups; each group shares 1 key and 1 value head (g KV heads total, reducing KV-cache by $h/g\times$ with near-MHA accuracy).

- **Positional Encodings:**
  - **RoPE (Rotary Position Embedding):** Applies a position-dependent rotation matrix $R^d_{\Theta,m}$ to Q and K such that the inner product $\langle R_m q, R_n k \rangle$ depends **only on relative distance** $(m - n)$. Decays naturally with distance and does not affect V.

5.2 LLM Memory Math, Fine-Tuning (LoRA), Alignment & Quantization

- **Master the 3 Memory Formulas for LLMs (Guaranteed Calculation Style):**
  1. **Inference Weights Memory:** Params $\times$ Bytes per param (FP32 = 4B, FP16/BF16 = 2B, INT8 = 1B, INT4 = 0.5B). *Example: A 7B model in FP16 takes* $7 \times 2 = \mathbf{14}$ GB *just for weights.*
  2. **KV-Cache Memory Formula:**

$$\text{KV Cache (Bytes)} = 2 \times B \times S \times L \times n_{\text{kv_heads}} \times d_{\text{head}} \times \text{Bytes_per_precision}$$

(Where 2 is for K and V, B = batch size, S = sequence length, L = number of layers).

3. **Mixed-Precision Training Optimizer Memory (AdamW with FP16/BF16):**
   - Per parameter Ψ: FP16 weights (2B) + FP16 gradients (2B) + FP32 master weights (4B) + FP32 Adam 1st moment m (4B) + FP32 Adam 2nd moment v (4B) = $\mathbf{16\Psi}$ Bytes (plus activations!).

- **LoRA (Low-Rank Adaptation):**
  - Freezes pretrained weight $W_0 \in \mathbb{R}^{d \times k}$ and injects trainable low-rank matrices $W = W_0 + \frac{\alpha}{r} BA$, where $A \in \mathbb{R}^{r \times k}$ and $B \in \mathbb{R}^{d \times r}$ with rank $r \ll \min(d, k)$.
  - **Trainable Parameters:** $r \times (d + k)$ instead of $d \times k$.
  - **Initialization Trap (Classic MCQ):** A is initialized with random Gaussian $\mathcal{N}(0, \sigma^2)$ and B **is initialized to** 0 so that $\Delta W = BA = 0$ at the start of training!
  - **Inference Latency:** Zero extra inference latency because BA can be **merged directly into** W_0 ($W_{\text{merged}} = W_0 + \frac{\alpha}{r} BA$) after training.
- **Sampling & Decoding Hyperparameters:**
  - **Temperature (T):** Scales logits before softmax: $P_i = \frac{\exp(z_i / T)}{\sum_j \exp(z_j / T)}$.
    - As $T \to 0$: distribution approaches a one-hot argmax (**greedy decoding**, deterministic, repetitive).
    - As $T \to \infty$: distribution approaches **uniform** over vocabulary (maximum entropy/randomness).
  - **Top-k vs Top-p (Nucleus Sampling):** Top-k keeps fixed k highest-probability tokens; Top-p dynamically keeps the **smallest set of top tokens whose cumulative probability** $\geq p$ (adapting naturally when the model is confident vs uncertain).
- **Quantization, Numeric Formats & Knowledge Distillation (Edge AI Essentials):**
  - **FP16 vs BF16:**
    - FP16: 1 sign bit, **5 exponent bits**, 10 mantissa bits $\implies$ high precision, narrow dynamic range (prone to gradient underflow/overflow; requires **Dynamic Loss Scaling**).
    - BF16 (Brain Float 16): 1 sign bit, **8 exponent bits** (same as FP32!), 7 mantissa bits $\implies$ same dynamic range as FP32, no loss scaling needed.
  - **Uniform Affine vs Symmetric Quantization:**
    - Affine (Asymmetric): $x_q = \text{clamp}\left(\text{round}\left(\frac{x}{S}\right) + Z, q_{\min}, q_{\max}\right)$, where Scale $S = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}$ and Zero-Point Z maps real 0.0 to an exact integer.
    - Symmetric: Sets $Z = 0$ and uses $S = \frac{\max(|x_{\min}|, |x_{\max}|)}{q_{\max}}$ (eliminates zero-point cross-terms during matrix multiplication on hardware!).
  - **PTQ vs QAT:**
    - **PTQ (Post-Training Quantization):** Calibrates scales on a small unlabeled calibration set without retraining (e.g., GPTQ, AWQ).
    - **QAT (Quantization-Aware Training):** Inserts fake-quantization nodes during training; uses the **Straight-Through Estimator (STE)** $\frac{\partial \text{round}(x)}{\partial x} = 1$ during backpropagation because round$(\cdot)$ has zero derivative almost everywhere.
  - **Knowledge Distillation:**
    - Student minimizes $\mathcal{L} = (1 - \alpha)\mathcal{L}_{\text{CE}}(y, \sigma(z_s)) + \alpha T^2 D_{\text{KL}}\left(\sigma(z_t / T) \parallel \sigma(z_s / T)\right)$.

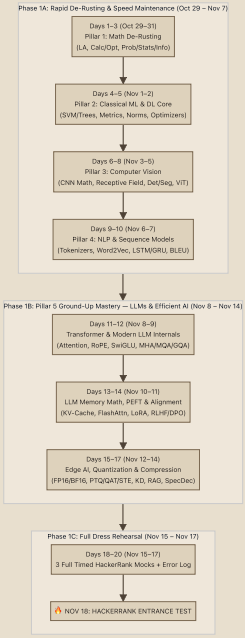
- *Why multiply by T^2?* Softening logits by $1/T$ scales the gradients of the KL loss by $1/T^2$; multiplying by T^2 preserves gradient magnitude relative to the hard-label loss!

---

3. Personalized 20-Day Execution Calendar (Oct 29 – Nov 18, 2026)

↑ Mục Lục

> ⚡ **IMPORTANT**
>
> **Strategic Allocation Based on Your Profile:**
>
> - **Pillars 1–4 (Math, ML/DL, CV, NLP):** You already have the conceptual foundation and just need **aggressive de-rusting & speed maintenance**. We compress Pillars 1–4 into **Days 1–10 (Oct 29 – Nov 7)** using high-density formula sheets and timed calculation/trap drills.
> - **Pillar 5 (Transformers, LLMs & Qualcomm Efficient/On-Device AI):** Since this is a new area for you —and a flagship focus of the **Qualcomm AI Residency Engineering Track**—we allocate a dedicated **7-Day Ground-Up Deep Dive (Days 11–17: Nov 8 – Nov 14)**, plus 30-minute daily priming during Week 1.
> - **Final Dress Rehearsal (Days 18–20: Nov 15 – Nov 17):** Full-length timed HackerRank simulations and Error Log eradication before **Exam Day on Nov 18**.



Day-by-Day Master Schedule (Oct 29 – Nov 18)

Day & Date	Track & Pillar	Core Topics to Master (De-Rust / Deep Dive)	Daily Timed MCQ Drill Target
Day 1 *Thu, Oct 29*	**Pillar 1A: Linear Algebra**	Trace/Det eigenvalue hacks, Rank-Nullity, Invertibility, Symmetric/PSD matrices, SVD ($U\Sigma V^T$), PCA variance explained, Eckart-Young, Orthogonal projections ($P^2 = P$).	15 Linear Algebra MCQs (`mcq_simulator.html`)

Day & Date	Track & Pillar	Core Topics to Master (De-Rust / Deep Dive)	Daily Timed MCQ Drill Target
Day 2 *Fri, Oct 30*	**Pillar 1B: Calculus & Optimization**	Matrix derivatives ($\nabla_x(x^TAx)$, $\nabla_w\|Xw-y\|^2$), Softmax+CE & Sigmoid+BCE backprop ($\hat{y}-y$), Hessian & saddle points, Convexity & Jensen's inequality, KKT, GD step-size bound ($\eta < 2/L$), Momentum/Adam/AdamW.	15 Calculus & Optimization MCQs
Day 3 *Sat, Oct 31*	**Pillar 1C: Prob, Stats & Info Theory**	Bayes' theorem & medical/spam paradoxes, Expectation/Variance/Covariance linearity, Gaussian/Poisson/Binomial/Beta, MLE vs MAP (Gaussian $\rightarrow$ L2, Laplace $\rightarrow$ L1), Entropy, Cross-Entropy, KL Divergence asymmetry.	15 Prob/Stats MCQs + **30-Q Pillar 1 Timed Sprint (45m)**
Day 4 *Sun, Nov 1*	**Pillar 2A: Classical ML & Metrics**	Ridge closed-form, Logistic Regression, SVM margin ($2/\|w\|$) & slack C, Decision Trees (Gini/Entropy), Bagging (RF variance reduction, OOB $1/e$) vs Boosting (XGBoost bias reduction), K-Means/GMM, ROC-AUC vs PR-AUC, F1.	25 Classical ML & Metrics MCQs
Day 5 *Mon, Nov 2*	**Pillar 2B: Deep Learning Core**	Sigmoid/Tanh/ReLU/GELU derivatives & saturation, **BatchNorm vs LayerNorm vs RMSNorm vs GroupNorm** (shapes, params, train vs eval), Inverted Dropout ($1/(1-p)$), Xavier vs Kaiming init, L1 sparsity subgradients.	25 Deep Learning Core MCQs
Day 6 *Tue, Nov 3*	**Pillar 3A: CNN Arithmetic & Efficiency**	Conv2D / Transposed Conv / Dilated Conv output size formulas, parameter counts (with/without BN bias), MACs/FLOPs, 1×1 bottlenecks, **Depthwise Separable Conv** reduction ratio ($\frac{1}{C_{\text{out}}}+\frac{1}{K^2}$).	20 CNN Calculation MCQs
Day 7 *Wed, Nov 4*	**Pillar 3B: Receptive Field, Detection & Seg**	Iterative Receptive Field formula ($RF_l = RF_{l-1} + (K_l - 1)J_{l-1}$), ResNet skip-connection gradients, IoU, NMS step-by-step, Anchor boxes, Focal Loss, RoI Pooling vs RoI Align, U-Net skip concatenation vs FCN addition.	20 Detection & Segmentation MCQs
Day 8 *Thu, Nov 5*	**Pillar 3C: Vision Transformers & Multimodal**	ViT patchification ($N = HW/P^2$), `[CLS]` token, positional embeddings, attention complexity on high-res images, CNN vs ViT inductive biases, Swin Transformer shifted windows, CLIP InfoNCE contrastive loss ($N \times N$ matrix).	20 ViT & Multimodal MCQs
Day 9 *Fri, Nov 6*	**Pillar 4A: Tokenization & Embeddings**	BPE (frequency merge) vs WordPiece (likelihood/PMI merge) vs Unigram LM, TF-IDF edge cases, Word2Vec (CBOW vs Skip-gram, Negative Sampling $3/4$ power, Hierarchical Softmax), GloVe co-occurrence matrix.	20 Tokenization & Embedding MCQs
Day 10 *Sat, Nov 7*	**Pillar 4B: Sequence Models & NLP Metrics**	RNN vanishing/exploding gradients (singular values of W_{hh}, gradient clipping), LSTM (4 gates, additive cell state c_t) vs GRU (3 gates) exact parameter counts, Bahdanau vs Luong attention, Beam Search, Perplexity ($\exp(\text{CE})$), BLEU vs ROUGE.	20 Sequence Model MCQs + **Midpoint Pillars 1–4 Mixed Review**
Day 11 *Sun, Nov 8*	**Pillar 5A (Deep Dive): Transformer Foundations**	From-scratch derivation of Scaled Dot-Product Attention ($\text{Var}(q\cdot k) = d_k$ proof), Causal Masking ($-\infty$), Multi-Head Attention shapes & $4D^2$ params, FFN $8D^2$ params, $O(N^2D + ND^2)$ crossover, Pre-LN vs Post-LN gradient flow, BERT vs GPT vs T5.	20 Transformer Mechanics MCQs

Day & Date	Track & Pillar	Core Topics to Master (De-Rust / Deep Dive)	Daily Timed MCQ Drill Target
Day 12 *Mon, Nov 9*	**Pillar 5B (Deep Dive): Modern LLM Architectures**	LLaMA / Qwen / Mistral architecture internals: **RoPE (Rotary Position Embedding)** relative distance properties, ALiBi, **SwiGLU** ($3D \cdot D_{\text{ff}}$ params), **RMSNorm**, **MHA vs MQA vs GQA** (KV head sharing math).	20 Modern LLM Architecture MCQs
Day 13 *Tue, Nov 10*	**Pillar 5C (Deep Dive): LLM Memory, KV-Cache & Inference**	Prefill (compute-bound) vs Decode (memory-bandwidth-bound), **Exact KV-Cache Byte Formula**, **AdamW Mixed-Precision** 16Ψ **Byte Rule**, Activation Checkpointing, **FlashAttention** (tiling + online softmax, $O(N)$ HBM), Temperature / Top-k / Top-p.	20 LLM Memory & Inference Calculation MCQs
Day 14 *Wed, Nov 11*	**Pillar 5D (Deep Dive): PEFT (LoRA) & Alignment**	**LoRA** ($W_0 + \frac{\alpha}{r}BA$, why $A \sim \mathcal{N}, B = 0$, parameter count $r(d+k)$, zero inference overhead via merging), QLoRA (4-bit NormalFloat + Double Quantization), SFT, **RLHF (Reward Model + PPO + KL penalty)** vs **DPO**.	20 PEFT & Alignment MCQs
Day 15 *Thu, Nov 12*	**Pillar 5E (Deep Dive): Quantization & Numeric Formats**	**FP32 vs FP16 vs BF16** bit layouts (sign/exponent/mantissa) & Dynamic Loss Scaling, **Affine (S, Z) vs Symmetric ($Z = 0$) Quantization** hand calculations, Per-Tensor vs Per-Channel vs Group-wise, **PTQ (GPTQ, AWQ, SmoothQuant)** vs **QAT (Straight-Through Estimator STE)**.	25 Quantization & Numeric Format MCQs
Day 16 *Fri, Nov 13*	**Pillar 5F (Deep Dive): Distillation, Pruning, Speculative Decoding & RAG**	**Knowledge Distillation** (T^2 gradient scaling proof), Unstructured vs Structured Pruning vs 2:4 Sparse Tensor Cores, **Speculative Decoding** (lossless rejection sampling math), **RAG** (Bi-Encoder MIPS/HNSW retrieval vs Cross-Encoder reranking).	20 Compression, SpecDec & RAG MCQs
Day 17 *Sat, Nov 14*	**Pillar 5G: Complete LLM & Efficient AI Gauntlet**	End-to-end synthesis of Pillar 5 formulas (params, FLOPs, KV-cache, quantization scales, LoRA, attention variants).	**50-Question Timed Pillar 5 Boss Exam (65m)**
Day 18 *Sun, Nov 15*	**Mock Exam #1 + Weak-Spot Remediation**	Full-length HackerRank Simulation #1 across all 5 pillars (Math, ML/DL, CV, NLP, LLMs/Edge AI). Deep post-mortem of every missed question.	**50-Question Full Mock Exam #1 (60m)**
Day 19 *Mon, Nov 16*	**Mock Exam #2 + Multi-Select Trap Eradication**	Full-length HackerRank Simulation #2 (heavy multi-select & subtle boundary cases) + re-solving 100% of logged mistakes in `mcq_simulator.html`.	**50-Question Full Mock Exam #2 (60m)**
Day 20 *Tue, Nov 17*	**Final 90-Second Formula & Invariant Review**	Light review of the Master Formula Cheat Sheet, mental math warmups, HackerRank environment check (camera, browser, scratch paper prep), early rest.	20 Rapid-Fire Speed Drills (20m)

Day & Date	Track & Pillar	Core Topics to Master (De-Rust / Deep Dive)	Daily Timed MCQ Drill Target
EXAM DAY *Wed, Nov 18*	🔥 **QUALCOMM AI RESIDENCY ENTRANCE TEST**	Execute the 2-pass strategy: Pass 1 answers all < 60s conceptual/invariant questions; Pass 2 tackles heavy multi-step calculations.	**Crush the Exam**

---

4. Active `QARE_prep` Workspace Architecture

↑ Mục Lục

```text
QARE_prep/
├── phase1_mcq_prep/
│   ├── mcq_simulator.html          # Interactive HackerRank MCQ Simulator (Practice + Timed Procto
│   ├── 01_math_foundations/        # Pillar 1 De-Rusting Guide (LA, Calc/Opt, Prob/Stats/Info Theo
│   ├── 02_ml_dl_core/              # Pillar 2 De-Rusting Guide (Classical ML, Metrics, Norms, Opti
│   ├── 03_computer_vision/         # Pillar 3 De-Rusting Guide (CNN Math, Receptive Field, Det/Seg
│   ├── 04_nlp_fundamentals/        # Pillar 4 De-Rusting Guide (Tokenizers, Embeddings, LSTM/GRU,
│   ├── 05_llm_and_efficient_ai/    # Pillar 5 Zero-to-Hero Manual (Transformers, LLMs, KV-Cache, L
│   └── mock_exams/                 # Full-length timed mock exam banks
└── phase2_coding_interview/        # Reserved for Phase 2 (DSA + NumPy/PyTorch Vectorization)
```